

PRA-MLX: Query-Addressed Memory beside Prompt and Rotating KV Caches

Jorge Simão
EInnovator R&D

August 2026

Abstract

Apple Silicon changes the physical cost model of long-context inference: weights, cache arrays, and host code share unified memory. It does not remove the logical question of which retained evidence should be active. We implement an MLX adapter boundary for Progressive Retrieval Attention (PRA), measure selected-text coexistence with the MLX-LM nearest-prefix cache, and test whether a rotating sequential cache can serve as a simpler archive. On an Apple M5 with the 4-bit Qwen3-0.6B checkpoint, selected and full context recover a fixed answer in 5/5 server requests; selected context uses 365 rather than 1,577 prompt tokens. Warm requests reuse 364 and 1,576 cached tokens, respectively, while the server prompt cache grows to 0.28 GB across five distinct sequences. In a separate five-seed Python-API experiment, full sequential KV recovers the answer in 5/5 cases with 112 MB of cache. Rotating KV at 64–512 tokens recovers 0/5. Reintroducing the selected archive record reaches 2/5 only at 64 tokens and 0/5 at larger capacities. We then implement native selected K/V at every layer. Across five seeds it is bit-exact to ordinary split-cache execution, recovers 5/5 answers, and remains 5/5 when local sequential K/V rotates at 64 tokens. Thus native PRA restores archived evidence without making it visible prompt text; broader workloads, quantized detail, and concurrency remain open.

1 Introduction

MLX provides an efficient array framework and model runtime for Apple Silicon [1]. MLX-LM adds explicit prompt caches, rotating K/V, optional cache quantization, batching, and an OpenAI-like server. These tools make sequential context reuse practical in unified memory. PRA addresses a different object:

large retained memory $\rightarrow$ query-selected region $\rightarrow$ active native detail.

Unified memory may reduce copy overhead, but it does not make every retained token free to index, keep, or attend to.

This paper studies three questions. First, can selected context coexist with MLX’s exact prompt cache? Second, can a bounded rotating cache plus a selected archive record recover old evidence? Third, what software boundary is needed before an MLX adapter may claim native PRA K/V rather than selected visible text?

2 Cache Taxonomy

We keep three mechanisms distinct:

Prompt cache

exact reusable prompt-prefix K/V, retained by MLX-LM.

Rotating cache

bounded recent sequential K/V, with old positions evicted as the prompt advances.

PRA memory

query-addressed non-prefix resources whose address state, native detail, and active selection have separate lifecycles.

Prompt-cache bytes are physical retained state. PRA backing may be much larger than active detail. Rotating capacity bounds direct context but does not create an address for evicted evidence.

Table 1: MLX-LM server smoke. TTFT is milliseconds; warm cached is the mean over repeats two through five.

Condition	Exact	Prompt	Cold TTFT	Warm TTFT	Warm cached
None	0%	24	523.6	505.2	23
Prefix	0%	333	446.4	355.2	332
Selected	100%	56	415.9	355.0	55
Prefix + selected	100%	365	432.0	352.3	364
Full context	100%	1577	565.4	351.8	1576

3 Implementation

The engine-neutral namespace in `src/pra_hf/engine_memory.py` identifies a block by tenant/session scope, resource/version, token interval, consumer layer, immutable model revision, dtype/layout, materialization profile, and position policy. Its residency state machine records indexed-only, off-device, prefetching, resident, pinned, evictable, and invalid states. In MLX, the tier name denotes array residency or backing policy within unified memory; we do not mislabel it as CUDA host-to-device transfer.

`src/pra_mlx/adapter.py` exposes the HTTP server as E0. It advertises automatic prefix caching and text fallback, but no native K/V. An in-process `MLXNativeExecutor` is required before the adapter advertises E2, selected interval materialization, logical resource deltas, or explicit prompt cache handles. `src/pra_mlx/native.py` now provides that executor.

The design choice is important. Precomputing a prompt cache from selected text still makes that text a sequential prefix. A native executor must instead preserve the selected region’s identity and positional policy while combining its K/V with local K/V under correct model attention.

The implementation encodes immutable resource text once and retains post-RoPE K/V in $[B, H_{kv}, T, D]$ form for every layer. A request-local cache returns *selected memory plus local sequential K/V* to each attention module and constructs a mask in which every query sees selected memory while local keys remain causal. Query positions advance from the source extent; selected text never appears in the visible prompt. The residency manager deduplicates materialization, pins blocks for request lifetime, and evicts only unpinned detail under a bounded byte budget.

4 Environment

The host was an Apple M5 MacBook Pro with 10 CPU and 10 GPU cores, 16 GB unified memory, Metal 4, and macOS 26.4.1. The environment used Python 3.12.7, MLX 0.32.0, and mlx-lm 0.31.3. The model was `mlx-community/Qwen3-0.6B-4bit` at revision `73e3e38d981303bc594367cd910ea6eb48349da8`. The HTTP server retained up to 16 prompt-cache entries. It emits an explicit warning that its basic security checks are not production-grade; we treat it as an experimental deployment baseline.

5 Server Prompt-Cache Experiment

The shared fixed task contains a stable 309-token prefix, one authoritative codeword record, twelve distractor records, and a short query. Five conditions separate no memory, prefix only, selected memory only, prefix plus selected memory, and full context. Each is repeated five times. The server reports cached tokens in the streaming usage object, and SSE events determine TTFT.

Table 1 shows correct controls: selected and full conditions are 5/5, while conditions without evidence are 0/5. Prefix plus selected memory uses 365 tokens and full context uses 1,577, again a 76.9% visible reduction. Warm selected requests reuse 364 tokens; warm full requests reuse 1,576. The server’s nearest-prefix cache therefore composes cleanly with the selected-text baseline.

Warm TTFT is nearly flat, 352.3 versus 351.8 ms, because exact prompt caches remove most repeated prefill work in both conditions. This is a useful negative latency result: after a perfect repeat cache hit, reducing visible input does not improve warm TTFT on this setup. Physical retention still differs. The log shows five distinct prompt-cache sequences occupying about 0.28 GB at the end of the matrix.

Table 2: Five-seed controlled rotating-cache experiment. Cache MB is the sum of layer-cache `nbytes`; latency is the mean completion time.

Condition	K/V tokens	Exact	Cache MB	Latency ms
Full sequential K/V	full	100%	112.0	468.9
Rotating only	64	0%	7.0	706.6
Rotating only	128	0%	14.0	730.2
Rotating only	256	0%	28.0	722.1
Rotating only	512	0%	56.0	721.2
Rotating + selected archive	64	40%	7.0	704.7
Rotating + selected archive	128	0%	14.0	736.2
Rotating + selected archive	256	0%	28.0	713.1
Rotating + selected archive	512	0%	56.0	732.2

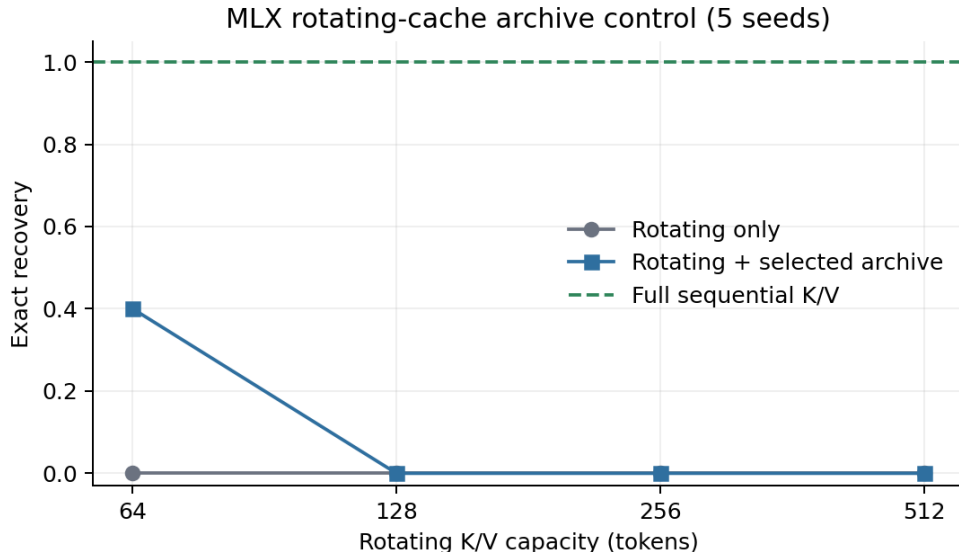


Figure 1: A bounded rotating cache is not a reliable archive in this control. Selected late text recovers two of five cases only at the smallest capacity.

6 Rotating Cache and Selected Archive

The Python-API control uses five deterministic filler permutations. One authoritative four-digit fact appears near the beginning of a roughly thousand-token prompt. We compare full sequential K/V with rotating capacities 64, 128, 256, and 512. For each rotating capacity, a second condition reintroduces the authoritative record immediately before the final question. The answer metric accepts harmless formatting but requires the correct digits.

Full sequential K/V is 5/5 with 112 MB of cache. Rotating-only is 0/5 at every capacity while reducing layer-cache storage from 112 MB to 7, 14, 28, and 56 MB. Selected archive text reaches 2/5 at 64 and 0/5 at larger capacities. Mean completion latency is 469 ms for full K/V and approximately 705–736 ms for bounded conditions. The generated outputs show chat-format and answer-path instability, not merely loss of the early fact.

This non-monotonic result should not be read as evidence that smaller cache is better. It indicates that the direct rotating-cache intervention changes more than evidence availability for this model and prompt. The selected-text fallback does not restore stable computation. A native PRA executor must retain local sequential semantics while adding selected memory separately; the next section evaluates that mechanism directly.

Peak process memory remains about 1.01–1.07 GB across these conditions despite the 7–112 MB cache range. Model and runtime allocations dominate on this small checkpoint, so layer-cache bytes are the interpretable economy metric. Larger models and memory pressure are required before process-level savings claims.

Table 3: Five-seed native selected-K/V control. Latency excludes the separately reported one-time native encoding cost. Native MB is all-layer selected K/V.

Condition	Exact	Latency ms	Native MB	Max error
Ordinary split cache	100%	187.0	0.0	0
Native selected K/V	100%	166.3	15.4	0
Native + rotating local 64	100%	215.5	15.4	0

Table 4: Cross-model native selected-K/V replication. Native and rotating columns report exact recovery over five seeds; error is zero for every row.

Model	Seeds	Native exact	Rotating exact	Native MB
Qwen3 0.6B 4-bit	5	100%	100%	15.42
Llama 3.2 1B 4-bit	5	100%	100%	4.34
Gemma 3 1B 4-bit	5	0%	0%	3.58

7 Native Selected-K/V Experiment

The source contains 141 tokens. Ordinary split-cache and native selected-K/V produce identical logits, identical argmax tokens, and 5/5 exact recovery. Native selected execution averages 166.3 ms after a separately measured 55.4 ms one-time encoding step and retains 15.4 MB of all-layer K/V. Most importantly, replacing the local sequential cache by a 64-token rotating cache still recovers 5/5. The selected archive is therefore independent of local rotation rather than late visible text competing inside that bounded cache.

Table 4 repeats the mechanism on Llama 3.2 1B and Gemma 3 1B. Llama achieves 5/5 native and rotating-local recovery with exact split-cache logits while retaining 4.34 MB. Gemma also has zero logit error and matching argmax decisions, but both its ordinary and native conditions are 0/5 on this particular answer-format task. Gemma therefore establishes mechanism parity, not answer recovery. Together the runs cover three attention topologies and tokenizers; they are not yet natural-workload results.

8 Promotion Gates

Gate	Status	Evidence
environment	closed	MLX Metal generation and server repeat reliably
prompt-cache coexistence	smoke closed	364/365 selected tokens reused warm
rotating-cache control	controlled closed	five seeds, four capacities
native semantic parity	controlled closed	five seeds, zero logit error, 5/5 recovery
rotating plus native	controlled closed	5/5 with 64-token local cache
lazy native region	prototype closed	selected resources encode on first selection
quantized PRA detail	open	model weights are 4-bit; PRA K/V unmeasured
concurrency/pressure	open	one request at a time, one small model

9 Falsification and Limitations

Native PRA-MLX is unnecessary if selected visible text plus ordinary MLX caching matches its quality, memory, and latency on realistic long-memory workloads. The server smoke makes that baseline competitive. Conversely, the rotating control shows that bounded sequential KV is not automatically a PRA archive.

The study uses one 4-bit model, one machine, a synthetic server task, and a controlled five-seed rotating fixture. The server experiment has only five repeats per condition, so p95/p99 are not reported. No native Q/K index, detail-layer profile, K/V quantization, persisted PRA cache, concurrency, or memory-pressure failure is measured. The rotating intervention also changes effective local context and can perturb chat-template semantics.

10 Related Work

MLX targets efficient and flexible machine learning on Apple Silicon [1]. Prompt caching and rotating K/V optimize sequential state; cache quantization reduces its physical size. Retrieval and context compression reduce visible input. PRA instead assigns stable identity to retained non-prefix memory and separates cheap addressing from active native detail. The mechanisms can be combined, but none is a semantic substitute for the others.

11 Conclusion

MLX prompt caching and selected context compose cleanly: the fixed answer is preserved with 76.9% fewer visible tokens, and repeated selected prompts reuse 364 of 365 tokens. Rotating K/V provides large layer-cache savings but fails as an archive in the five-seed control; late selected text does not reliably repair it. The in-process native executor passes the missing mechanism gate: selected non-prefix K/V is bit-exact to ordinary split-cache execution and preserves 5/5 recovery with a rotating local cache. PRA-MLX is therefore a working native prototype. Quantized detail, calibrated layer profiles, additional models, persistence, and concurrent memory pressure remain necessary before production claims.

References

- [1] A. Hannun et al. MLX: Efficient and Flexible Machine Learning on Apple Silicon. 2023.